Journal of Systems Architecture 174 (2026) 103735

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

ZJC: Constructing fully local repair in erasure codes for distributed cloud storage

Zijian Zhou [a,b,c], Xiaoheng Deng [a,b,*], Xinjun Pei [c], Yunlong Zhao [a,b], Yurong Qian [c], Shaohua Wan [d], Kaiping Xue [e]

[a] School of Electronic Information, Central South University, Changsha, 410083, China
[b] Shenzhen Research Institute, Central South University, Shenzhen, 518000, China
[c] School of Software, Xinjiang University, Urumqi, 830091, China
[d] The Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen, 518000, China
[e] Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei, 230027, China

ARTICLE INFO

Keywords:
Erasure code
Encoding theory
Storage system
Distributed system

ABSTRACT

Achieving efficient repair with minimal redundancy remains a long-standing challenge for distributed cloud storage. Existing erasure codes such as Reed–Solomon (RS) and Local Reconstruction Codes (LRC) rely on global repair, which causes heavy repair bandwidth consumption and high node construction overhead. This paper presents ZJ Codes (ZJC), a novel erasure coding scheme that achieves fully local repair while maintaining strong availability. ZJC introduces an interleaved local-group structure in which each data block participates in two local parity groups, thus providing two independent recovery paths without any global parity. A new storage overhead model is further proposed, incorporating node construction cost into redundancy evaluation. We analytically prove that ZJC achieves lower storage overhead and constant repair bandwidth with only two helper blocks per repair. Both RS-based and XOR-based implementations are developed and experimentally deployed in a distributed system. Results show that ZJC improves encoding efficiency by up to 87.5% and recovery throughput by 96.5% compared with Reed–Solomon codes at equivalent redundancy. Compared to LRC, ZJC achieves encoding improvements of 62.3% and 86.5%, respectively. RS-based and Xor-based ZJC show recovery improvements of 53.9% and 96.5% compare with RS, respectively. Additionally, Xor-based ZJC outperforms LRC and ZJC RS-based by 77.5% and 90.8%, respectively. These results demonstrate that ZJC effectively reconciles the long-standing trade-off between low redundancy and high repair efficiency in distributed cloud storage.

1. Introduction

High data availability is the cornerstone of modern cloud and edge storage systems. As data volumes continue to surge, ensuring fault-tolerant and cost-efficient storage becomes increasingly challenging. Node or disk failures, once sporadic, are now routine in hyperscale infrastructures, and the cost of recovery has become a key factor in system design. Classical approaches such as redundant backups (RB) offer high reliability but suffer from excessive storage cost [1,2]. Erasure codes (EC), on the other hand, improve storage efficiency by generating parity blocks from k data blocks and distributing them across $n = k + m$ nodes [3,4]. Reed Solomon (RS) codes, as the most widely adopted EC, are Maximum Distance Separable (MDS) code that can tolerate any m failures [5,6]. However, their heavy finite-field computation and global repair bandwidth significantly increase recovery latency.

To address the inefficiency of RS codes, Microsoft Azure introduced Local Reconstruction Codes (LRC) [7], which utilizes partial data subsets to form *local groups*, thus reducing bandwidth consumption during recovering for blocks in the local groups. Particularly, the bandwidth consumption for reconstructing a block is defined as *repair bandwidth*. Further, the recovery pattern for local groups is named as *local repair*. In contrary, *global repair* refers to recover blocks through the *global parity* which is computated from all data blocks. Despite the remarkable efficiency gains introduced by *locality* [8,9], the pursuit of fully local repair remains an open challenge. Existing LRC constructions achieve only partial locality—while they reduce repair bandwidth for single-node failures, they still rely on global parity to ensure recoverability under multiple failures. This inherent dependency exposes a critical

* Corresponding author.
E-mail addresses: zijianzhou@stu.xju.edu.cn (Z. Zhou), dxh@csu.edu.cn (X. Deng), peixinjun@xju.edu.cn (X. Pei), zhaoyl741@csu.edu.cn (Y. Zhao), qyr@xju.edu.cn (Y. Qian), shaohua.wan@uestc.edu.cn (S. Wan), kpxue@ustc.edu.cn (K. Xue).

https://doi.org/10.1016/j.sysarc.2026.103735
Received 25 November 2025; Received in revised form 8 January 2026; Accepted 2 February 2026
Available online 7 February 2026

trade-off between fault tolerance and repair efficiency. Eliminating global repair entirely would not only minimize cross-cluster data transfer and computational complexity but also require a fundamentally new encoding structure capable of preserving recoverability within purely local groups. Therefore, understanding the importance of ***fully local repair*** and addressing the associated design challenges becomes essential for advancing erasure-coded storage systems.

The construction of erasure codes with fully local repair entails a fundamental challenge: maintaining high data availability while completely eliminating reliance on global repair. In LRC, local groups possess the Maximum Distance Separable (MDS) property, ensuring that any single block loss within a group can be recovered as long as the number of erasures does not exceed the MDS bound. The existence of global repair in LRC guarantees recoverability when this bound is violated. For example, two data blocks B_1 and B_2, together with a parity block P_1, form a local group. Any single block loss within this group can be reconstructed locally, whereas the simultaneous loss of two or more blocks requires assistance from global parity to restore the entire group.

Moreover, in conventional erasure-coded storage systems, data and parity blocks are distributed following the *one-block-per-node* (OBPN) principle, which simplifies fault isolation but incurs substantial node construction costs and redundant network utilization. In contrast, a locality-aware data placement strategy can substantially optimize the data layout. Because local repair inherently depends on a limited subset of neighboring blocks, co-locating these interdependent blocks within the same or adjacent nodes significantly reduces repair bandwidth and I/O latency, thereby fully exploiting the advantages of locality.

To address these challenges and realize fully local repair, this paper proposes ZJ-Codes (ZJC), a novel class of erasure codes that achieves complete repair locality through the integrated design of encoding and data placement. From an encoding perspective, ZJC forms local groups by interleaving data and parity dependencies so that each data block participates in two distinct local groups. This interleaved structure provides two independent recovery paths for every data block, thereby maintaining high data availability without invoking any global repair. Unlike conventional LRCs that rely on global parity when multiple erasures occur, ZJC ensures recoverability through purely localized operations, enabling repair processes to be confined entirely within local groups. Furthermore, ZJC supports both RS-based and XOR-based implementations, offering flexible computational approach. From a data layout perspective, ZJC redefines how blocks are physically placed across storage nodes. Instead of adhering to the traditional OBPN constraint, ZJC co-locates data and parity blocks on the same node, thereby halving the number of required storage nodes and substantially reducing storage overhead. To rigorously quantify these improvements, we establish a new storage overhead equation and introduce several analytical metrics that theoretically evaluate the trade-offs among node cost, redundancy rate, and fault tolerance. The contributions of this paper are summarized as follows:

1. We design ZJ-Codes (ZJC), the first erasure coding scheme that completely eliminates global repair and achieves recovery solely through local repair.
2. This paper fully exploits data locality by co-locating data and parity blocks on the same node during data placement. This design not only preserves fault tolerance but also reduces the number of required storage nodes, thereby lowering the construction and maintenance costs of cloud data center infrastructures.
3. Our data repair process is a combination of sequential and parallel repair, providing a basis for flexible bandwidth allocation and repair process regulation in an asynchronous manner.
4. ZJC can be implemented using multiple methods. We provide both RS-based and XOR-based implementations, and the experimental results demonstrate significant efficiency improvements compared with traditional RS and LRC codes. For reproducibility and further research, the source code has been made publicly available on GitHub.[1]

2. Background and related work

2.1. Background

2.1.1. Reed–Solomon codes

Reed–Solomon (RS) code [5] is both an error correcting code and an erasure code, which has a wide range of applications in storage and communication systems, e.g., CDs, QR Codes, etc. Reed–Solomon codes are usually represented by two parameters, which are the block length n and the message length k. Formally, presented as $RS(n,k)$. Assuming the message m_{ess} requires encoding, where k is the length of m_{ess} and the encoded length is n, $m = n-k$ parity data are added. The attributes of the RS code are as follows:

- *Property 1.* $RS(n,k)$ can detect errors at m locations but are unable to correct them. However, they possess the capability to both detect and correct errors at up to $\lfloor \frac{m}{2} \rfloor$ unknown locations. As an error correction code in such scenarios.
- *Property 2.* As an erasure code, $RS(n,k)$ becomes feasible to recover up to m errors with known error locations, even if they have been tampered with or erased.

2.1.2. Local Reconstruction Codes

Local Reconstruction Codes (LRC) [7], represented as $LRC(k,l,r)$, is the first one who comes up with the local repair idea and greatly reduces the repair bandwidth. In the $LRC(k,l,r)$, the amount of data blocks is k, the numbers of local group is l and r is the amount of global parity blocks. Each local group has k/l data blocks and generates one local parity block. It has the following properties:

- *Property 1.* Arbitrary $r+1$ blocks is recoverable, but it is not the MDS code. It can repair $l+r$ failure if they are information-theoretically decodable.
- *Property 2.* One lost block can be recovered by the other blocks in its group, i.e. local repair, which reduces the repair bandwidth.

2.2. Related work

Large-scale storage systems increasingly adopt erasure coding (EC) to strike a balance between reliability and storage cost [1,3,4]. Prior research spans three major directions: (i) *coding-structure acceleration* to reduce computation, (ii) *repair-process optimization* to cut bandwidth/latency, and (iii) *multi-failure availability* from both systems and coding-theoretic perspectives. We position our fully local repair (two independent local paths per data block) and asynchronous bandwidth scheduling against these lines of work.

(1) Coding-structure acceleration. Classical RS codes [5] are maximum distance separable (MDS) but involve costly finite-field operations. Many studies have focused on optimizing the coding structure to reduce computation or improve locality. Early works such as Cauchy RS optimization [10] and GPU acceleration (G-CRS) [11] improved RS throughput via matrix reformulation and parallel decoding. In parallel, the idea of *locality* was formally introduced through Locally Repairable Codes (LRCs), which restrict repair to a small subset of blocks to reduce bandwidth. Gopalan et al. established the locality bound [8], and Papailiopoulos and Dimakis developed optimal constructions [9]. Microsoft Azure later deployed LRCs at system scale [7], confirming their practical advantages in cloud storage. To tolerate multiple erasures, Prakash et al. extended LRCs with sequential recovery for two erasures [12], while recent works proposed new optimal constructions balancing distance and repair cost [13]. These advances

[1] https://github.com/Zijian-Zhou/ZJC_Compare

form the foundation for modern locality-aware codes but still rely on a combination of local and global parity groups. Beyond locality, computational acceleration has evolved along hardware and XOR-only dimensions: Cerasure [14] and its extended version [15] design fast XOR-based ECs leveraging spatial locality and wide stripes; gPPM [16] generalizes parallel matrix operations for erasure codes; and Y. Uezato. demonstrated program-level optimizations for XOR-coded paths [17]. While these efforts either improve computation efficiency or introduce partial locality, our ZJC differs by achieving *fully local repair*: each data block belongs to two distinct local groups, providing two independent recovery paths without involving global parity, while also supporting RS- and XOR-based implementations for performance flexibility.

Repair-process optimization. Production experience in Azure (LRC) shows that local parity groups reduce single-block repair bandwidth [7]. Subsequent efforts emphasize layout and cross-failure-domain awareness: orthogonal-array layouts and mixed-code data placement [18,19] reduce cross-rack traffic and balance load; at enclosure scale, RAID+ provides deterministic and balanced distribution [20]; system implementations such as ESetStore realize fast recovery under EC in practice [21]. More recent work pushes parallelism and scheduling: Hu et al. introduce *combined locality* to address wide-stripe repair by systematically combining parity and topology locality [22]; RepairBoost models full-node repair with a repair DAG and boosts end-to-end rebuild throughput across codes and algorithms [23]; ParaRC exposes sub-packetization parallelism of MSR codes and deploys parallel repair on HDFS as middleware [24]. Beyond disks, ELECT integrates EC tiering into the LSM-tree stack [6], and stripeless data placement has been proposed for in-memory EC to avoid stripe-induced overheads [25]. Our method differs in that it *eliminates global parity involvement during recovery*: each data block belongs to two distinct local groups, yielding two independent local repair paths and enabling asynchronous (sequential/parallel) scheduling to flexibly trade bandwidth for latency.

Multi-failure availability and theory. While single-node failures dominate in frequency, multi-failure scenarios drive recent systems advances. Aggregation decoding and tree-based cooperative recovery demonstrate practical multi-loss speedups [26,27]; hybrid centralized/independent strategies accelerate multi-block repair [28]; cross-rack-aware and seek-efficient single-failure methods improve end-to-end performance and remain building blocks within larger repairs [29–32]. Fast proactive repair further reduces risk windows by parallelizing migration and reconstruction with bipartite matching over clusters [33]. From the coding-theory side, some works established locality bounds and constructions [8,9], including sequential-recovery locality for two erasures [12] that inspires dual repair paths in our design; new optimal LRC constructions also push code parameters and length [13]. System studies continue to re-evaluate MSR/network coding under practical blob workloads [34]. Compared with approaches that rely on both local and global parity or on bandwidth-optimal (but system-heavy) MSR, our ZJC ensures *fully local repair* under multiple failures by construction and supports asynchronous repair to provision bandwidth elastically.

3. Design of ZJ-Codes

This section formalizes the design of ZJ-Codes (ZJC). We first provide the information of all notations, definitions and theoretical metrics in this paper (Section 3.1). We then use a running example to motivate fully local repair and contrast ZJC with RS and LRC at the same redundancy rate (Section 3.2). Next, we present a uniform construction of ZJC including the encoding matrix, block placement, and two disjoint local repair paths per data block (Section 3.3). We finally discuss fault tolerance, repair bandwidth, computational complexity, and implement approaches (Section 3.4), and summarize the key properties (Section 3.5).

Table 1
Notations.

Notation	Meaning
B	Data block
P	Parity block
N	Storage node
n	The number of all block
k	The number of data block
m	The number of parity block
γ	Redundancy rate, $\gamma = \frac{n}{k}$
ϑ	The number of required storage node for a stripe
η	Our proposed storage overhead metric
ρ	Repair bandwidth
r	Locality
d_{min}	The minimum distance

Table 2
Theoretical comparison of RS(12,6), LRC(6,3,3), and ZJC(6) under the same redundancy $\gamma = 2$.

Encoding method	γ	η	r	$\rho_{\min}$	$\rho_{\max}$	ARC_1	ARC_2	NRC	DC	Computation complexity
$RS(12,6)$	2×	24×	6	6	6	6	≈ 1.09	12	6	High
$LRC(6,3,3)$	2×	24×	2	2	6	3	≈ 0.55	6	2	Middle
$ZJC(6)$	**2×**	**12×**	**2**	**2**	**2**	**2**	≈ 0.36	**4**	**2**	**Low**

3.1. Notations, definitions and metrics

Notations: the information of notations are given in Table 1.
Definitions:

1. Repair Bandwidth: The number of helper blocks needed to recover a lost block, formally, we noted it as ρ. For example, for $RS(12,6)$ and $LRC(6,3,3)$, $\rho_{RS} = \rho_{min} = \rho_{max} = 6$, $\rho_{LRC} = 2$ when block in a local group but $\rho_{LRC} = 6$ when block in a global group.
2. Stripe: It refers to a set of data blocks and parity blocks. Formally, given an (n,k) EC, a stripe S is the ordered tuple of all encoded symbols and data symbols, i.e. $S = \{B_1, B_2, \ldots, B_k, P_1, P_2, \ldots, P_m\}$
3. OBPN: It denotes the classical *one-block-per-node* placement.
4. Locality: It first introduced and formed in 2012 [8] and futher researched in 2014 [9]. Locality presents that lost of every data block only require r other blocks to recover the lost data block. Moreover, Papailiopoulos and Dimakis [9] proved the Singleton-like Bound for LRC and bridged the gap between the d_{min} and r:

$$d_{min} \le n - k - \lceil \frac{k}{r} \rceil + 2. \tag{1}$$

Metrics:

1. Redundancy: For an EC(n,k) code, the formulation of redundancy is:

$$\gamma = \frac{n}{k}. \tag{2}$$

2. Our New Storage Overhead Metric η: Classical storage overhead metrics focus on redundancy, but ignore the hidden cost of *provisioning storage nodes*. We denote the required number of nodes by ϑ. Let η be the overall storage overhead that jointly reflects redundancy and node provisioning:

$$\eta \triangleq \vartheta \cdot \gamma = \vartheta \cdot \frac{n}{k}. \tag{3}$$

Fewer nodes (smaller ϑ) reduce per-node fixed costs (rack slots, power/cooling, orchestration), while smaller γ reduces disk usage. Eq. (3) therefore enables *apples-to-apples* comparison across codes that achieve the same γ but require different numbers of nodes.

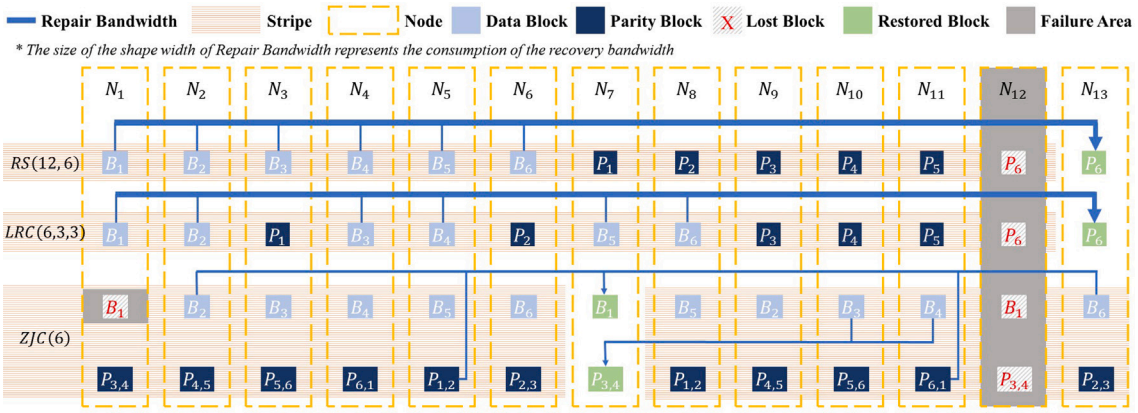


Fig. 1. Different EC Encode Structure and Blocks Distribution: It should be noticed that the two stripes for ZJC are used to illustrate the different recovery patterns for one block lost and one node failure. In fact, ZJC only need to store one stripe not two.

3. Average Repair Cost (ARC1): ARC1 proposed by the study [35] and is premised under the same failure possibility:
$$ARC_1 = \frac{\sum_{i=1}^{n} \rho(s_i)}{n}. \tag{4}$$
4. ARC2: For LRC, $\rho(B_i)$ is different for distinct block while same for RS. Research [36] introduced ARC2 to assess the repair cost for two blocks in a stripe:
$$ARC_2 = \frac{\sum_{i=1}^{n} \rho(s_i)}{\binom{n}{2}}. \tag{5}$$
5. Normalized Repair Cost (NRC): To take the cost of full node repair into consideration, research [37], based on ARC1, proposed NRC:
$$NRC = ARC_1 \times \gamma = \frac{\sum_{i=1}^{n} \rho(s_i)}{k}. \tag{6}$$
6. Average Degraded read cost (DC): DC only evaluate the repair cost of data blocks:
$$DC = \frac{\sum_{i=1}^{k} \rho(s_i)}{k}. \tag{7}$$
7. Locality: r.
8. Repair Bandwidth: ρ.

3.2. Motivation and running example

Fig. 1 (first two stripes) illustrates $RS(12,6)$ and $LRC(6,3,3)$ under OBPN with the same redundancy $\gamma = 2$. Table 2 compares the theoretical metrics for RS, LRC and ZJC. To better reading, we define the equation as follows:

$$\varsigma(x) = \sum_{i=1}^{x} \rho(s_i). \tag{8}$$

RS. $RS(12,6)$ produces m=6 parity blocks from k=6 data blocks. Any m erasures are repairable via global decoding. With OBPN, the 12 blocks reside on 12 nodes, hence $(\vartheta, \gamma, \eta) = (12, 2, 24)$ and $\rho_{\text{RS}} = 6$ for a single-block failure. $\varsigma_{RS}(n) = 12 \times 6$, $\varsigma_{RS}(k) = 6 \times 6$, $ARC_1 = \frac{\varsigma_{RS}(n)}{n} = \frac{12\times6}{12} = 6$, $ARC_2^{RS} = \frac{\varsigma_{RS}(n)}{6\times11} = \frac{12\times6}{6\times11} \approx 1.09$, $NRC_{RS} = \frac{\varsigma_{RS}(n)}{6} = \frac{12\times6}{6} = 12$, and $DC_{RS} = \frac{\varsigma_{RS}(k)}{6} = \frac{6\times6}{6} = 6$.

LRC. $LRC(6,3,3)$ partitions the six data blocks into l=3 local groups (each with a local parity) and adds three global parities. It prefers local repair when possible, giving ρ_{LRC}=2 for single-block failures. However, it is possible ρ_{LRC}=6 when a block lost which depicted in Fig. 1 (stripe 2). Under OBPN, $(\vartheta, \gamma, \eta) = (12, 2, 24)$. $\varsigma_{LRC}(n) = 9\times2+3\times6$, $\varsigma_{LRC}(k) = 6\times2$, $ARC_1 = \frac{\varsigma_{LRC}(n)}{n} = \frac{9\times2+3\times6}{12} = 3$, $ARC_2^{LRC} = \frac{\varsigma_{LRC}(n)}{6\times11} = \frac{9\times2+3\times6}{6\times11} \approx 0.55$, $NRC_{LRC} = \frac{\varsigma_{LRC}(n)}{6} = \frac{9\times2+3\times6}{6} = 6$, and $DC_{LRC} = \frac{\varsigma_{LRC}(k)}{6} = \frac{6\times2}{6} = 2$.

Observation. Both RS and LRC achieve the same redundancy but require 12 nodes when OBPN is enforced, leading to identical $\eta = 24\times$ by (3). This motivates a design that (i) preserves $\gamma = 2$, (ii) *reduces* ϑ by colocating different block classes on the same node, and (iii) still enables *fully local repair* with *constant* repair bandwidth.

ZJC at a glance. ZJC uses k nodes to store k data and k parity blocks by colocating one data block with one parity block per node. With $\gamma = 2$ and $\vartheta = k$, we obtain $\eta_{\text{ZJC}} = 2k$, i.e., at the same redundancy, ZJC halves the storage overhead compared with OBPN designs. For the concrete example $ZJC(6)$, $(\vartheta, \gamma, \eta) = (6, 2, 12)$ and $\rho_{\text{ZJC}} = 2$. In Fig. 1 (Stripe 3), we present two $ZJC(6)$ stripes to illustrate different ways to reconstruct a same data block and the recovery pattern for a parity block. $\varsigma_{ZJC}(n) = 12 \times 2$, $\varsigma_{ZJC}(k) = 6 \times 2$, $ARC_1 = \frac{\varsigma_{ZJC}(n)}{n} = \frac{12\times2}{12} = 2$, $ARC_2^{ZJC} = \frac{\varsigma_{ZJC}(n)}{6\times11} = \frac{12\times2}{6\times11} \approx 0.36$, $NRC_{ZJC} = \frac{\varsigma_{ZJC}(n)}{6} = \frac{12\times2}{6} = 4$, and $DC_{ZJC} = \frac{\varsigma_{ZJC}(k)}{6} = \frac{6\times2}{6} = 2$.

3.3. Uniform construction of ZJC

A $ZJC(k)$ instance takes k data blocks $\mathbf{D} = [B_1, \ldots, B_k]^T$ and produces k parity blocks $\mathbf{P} = [P_1, \ldots, P_k]^T$. Each parity combines *two* adjacent data blocks, and each data block participates in *two* local groups, yielding two independent local repair paths.

Encoding matrix. Let $\mathbf{E} = \mathbf{I}_k$ denote the identity mapping for systematic storage. Let $\mathbf{C} \in \{0,1\}^{k\times k}$ be the circulant matrix whose ith row has ones at columns i and i+1 (indices modulo k), i.e., each parity is an XOR/linear combination of two adjacent data blocks. We further introduce an *offset parameter* $\delta \in \{0, \ldots, k-1\}$, and let $\mathbf{F}$ be the $k \times k$ circulant permutation matrix that rotates a vector by δ positions. The parity-generation matrix is

$$\mathbf{G} \triangleq \mathbf{C}\,\mathbf{F}, \qquad \begin{bmatrix} \mathbf{D} \\ \mathbf{P} \end{bmatrix} = \begin{bmatrix} \mathbf{E} \\ \mathbf{G} \end{bmatrix} \mathbf{D}. \tag{9}$$

Concretely, the jth parity is $P_j \leftarrow B_j \oplus B_{j+1}$ after an index rotation by δ (all indices modulo k). The parity block can be also noted as $P_{j,j+1}$, in this form, it implicitly presents which two data blocks generate this parity block. The offset δ determines which parity is colocated with which data block during placement and is chosen to maximize recoverability. We use the same δ notations as in our figures and experiments. The discussion of $det(G)$ is provided at the Appendix, which is crucial for decoding.

Node placement (two-classes-per-node). We store one data and one parity on the same node:

$$(B_i,\ P_i) \longrightarrow N_i, \qquad i \in \{1, \ldots, k\}, \tag{10}$$

where the index of P_i is taken modulo k. This eliminates OBPN and uses exactly k nodes for $(\gamma = 2)$.

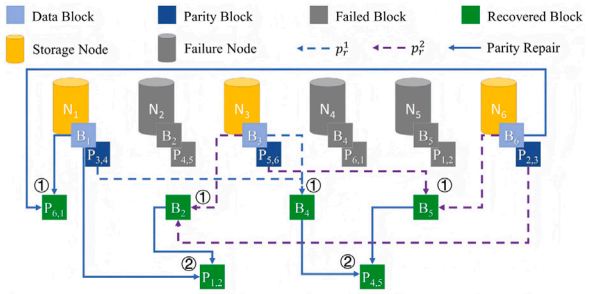


Fig. 2. Sequential and parallel repair.

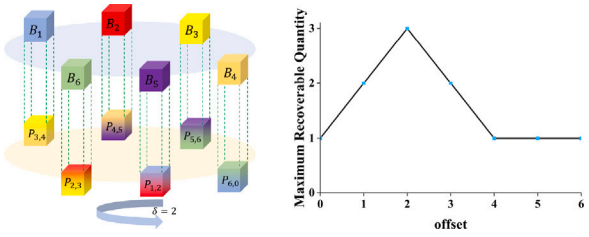


Fig. 3. 3D visualization of $ZJC(6)$ and the effect of distinct δ values.

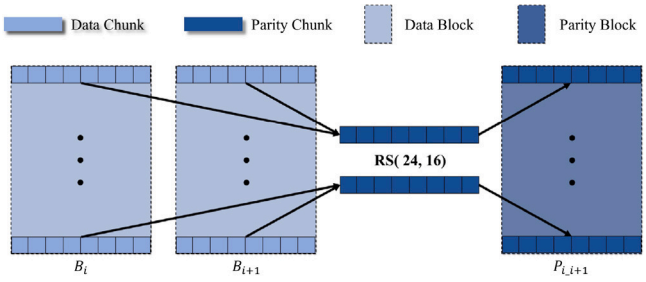


Fig. 4. An example of $ZJC(k)$ by $RS(24, 16)$.

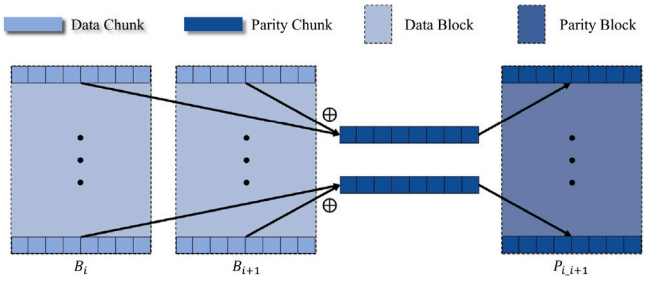


Fig. 5. An example of XOR-based $ZJC(k)$.

Two disjoint local repair paths per data block. By construction each data block belongs to two local groups:

$$p_r^{(1)}(i):\ B_i \leftarrow (B_{i-1},\ P_{i-1,i}), \tag{11}$$

$$p_r^{(2)}(i):\ B_i \leftarrow (B_{i+1},\ P_{i,i+1}). \tag{12}$$

Hence, for any single-block loss, there always exists a local path that uses *exactly two* helper blocks ($\rho = 2$). The two-path design is the core of *local-group interleaving checking (LGIC)*: every data symbol is protected by two partially overlapping local groups, providing availability without invoking any global repair. For example, Fig. 2 illustrates the recovery process of three nodes failure of six nodes. The recovery of block $\{B_2, B_5\}$ use the repair path $p_r^{(2)}$ while recovering B_4 by repair path $p_r^{(1)}$.

3.4. Fault tolerance and complexity

Single failure. If node N_i fails (losing B_i and P_i), B_i is repaired by either path in (11)–(12); P_i is then re-encoded from its two data parents. The repair bandwidth is constant $\rho_{\text{ZJC}} = 2$ for any single-block loss.

Multiple failures and asynchronous repair. When multiple nodes fail, ZJC proceeds in *asynchronous* rounds: any block whose local path is currently satisfied is repaired first; repairing those blocks often unlocks other paths in subsequent rounds. This enables flexible bandwidth scheduling without global decoding. For the running example in Fig. 2, the same failure pattern admits different valid repair orders, allowing systems to adapt to bandwidth and load. The recovery schemes presented in Fig. 2 are the results of the concurrence of the *sequential repair* and *parallel repair*. For instance, in the Fig. 2, the repair process proceeds sequentially across two stages (from No. 1 to No. 2). Within each stage, repairs are executed in parallel. Four No. 1 blocks are repaired simultaneously, and finally two No. 2 block repaired independently. Sequential repair refers to a recovery process in which lost blocks are repaired one after another, and the reconstruction of a later block depends on the results of previous repairs [38]. Parallel repair refers to a recovery process where all failed blocks are repaired independently and simultaneously, using only surviving (non-failed) blocks without any dependency on intermediate results.

Tolerable node failures. Let $R_{\max}$ denote the maximum number of failed nodes that remain repairable under (10) with a given offset δ.

We can visualize the layout of ZJC in a three-dimensional form, as depicted in Fig. 3. The data blocks and parity blocks are arranged on two concentric circular discs, where the blue disc represents data blocks and the yellow disc represents parity blocks. Each block corresponds to a point on its respective disc. By vertically aligning one data block with one parity block, a rectangle (green line) is formed, representing a storage node that colocates blocks.

To explore the effect of the offset parameter δ, we fix the data-block disc and counter-rotate the parity-block disc by δ positions. The number of rotations directly determines how data and parity pairs are distributed across nodes. In this geometric representation, the value of δ fundamentally controls the interleaving relationship between local groups, and consequently, the system's fault-tolerance capability.

As illustrated in Fig. 3, for $k = 6$, different δ values produce distinct connectivity patterns between data and parity discs. Empirically, the system achieves its maximum recoverable node failures $R_{\max} = 3$ when $\delta = 2$. This observation reveals that the offset δ is a critical design parameter: δ ensures maximizing fault tolerance while maintaining fully local repair.

For the family of $ZJC(k)$ considered in this paper, δ is determined by R_{max}: $\delta = R_{\max} - 1$ and we prove that (Appendix):

$$R_{\max} = \left\lfloor \frac{k}{2} \right\rfloor. \tag{13}$$

Thus ZJC sustains up to $\lfloor k/2 \rfloor$ node failures without invoking any global repair, while preserving $\rho = 2$ per recovered data block.

Computational complexity. Each parity symbol depends on two data symbols. Compared with RS/LRC global decoding (finite-field operations over k symbols), ZJC's local repair uses only two-symbol combinations (XOR or two-term finite-field ops), yielding much lower CPU cost and latency. In practice we provide both RS-based and XOR-based implementations, see Fig. 4–5.

The RS-based implementation aims to maximize the utilization of property 2 of RS code. To reach the property, we construct the $RS(3\zeta, 2\zeta)$ that arbitrary ζ chunk (i.e. k symbols of blocks) lost is recoverable. In a group of ZJC, there are two data blocks and one parity block. To construct a parity block, we sequentially sample ζ chunks

from each of two data blocks, resulting in a total of 2ζ data chunks. Based on these sampled chunks, ζ parity chunks are computed, which are then placed into the corresponding positions of the parity block. After all required data chunks have been sampled and processed, the parity chunks are arranged in order, thereby forming the final parity block. The Fig. 4 gives a $RS(24, 16)$ example in a local group, where $\zeta = 16$. In this instance, the size of the RS strip is 24 bytes. In this method, we require the RS parameter should be $n : k : m = 3 : 2 : 1$.

The XOR-based implementation is more flexible than the RS-based and uses a similar pattern. The XOR operation has the following property, $a \oplus b = c$, then $a = c \oplus b$ and $b = c \oplus a$ when a or b is lost. We can use *chunk by chunk* or *byte by byte* to calculate the parity block. Fig. 5 gives a *chunk by chunk* example in a local group, and the chunk size is 8 bytes.

3.5. Summary

$ZJC(k)$ produces k parity blocks from k data blocks; each parity combines two adjacent data blocks via a circulant construction, and each data block is covered by two local groups, yielding two disjoint repair paths. Blocks are placed with one data and one parity per node, using exactly k nodes at redundancy $\gamma = 2$, so $\eta = 2k$. ZJC achieves constant repair bandwidth $\rho = 2$ for any single-block failure and tolerates up to $R_{\max} = \lfloor k/2 \rfloor$ node failures via asynchronous local repair, while maintaining low computational cost and avoiding global decoding.

4. Example of *ZJC(6)*

To provide a concrete understanding of ZJC, we now instantiate the design presented in Section 3 with a specific example $ZJC(6)$, which encodes $k = 6$ data blocks into $n = 12$ total blocks. This example illustrates the structure of the encoding matrices, the resulting parity equations, the node layout, and the corresponding repair behaviors.

4.1. Encoding matrices

Let $\mathbf{D} = [B_1, B_2, B_3, B_4, B_5, B_6]^T$ denote the data vector. Following the general construction (9), ZJC employs two 6×6 circulant matrices: the base encoding matrix $\mathbf{C}$ and the offset (rotation) matrix $\mathbf{F}$.

For $ZJC(6)$, the base matrix $\mathbf{C}$ has two adjacent 1's in each row, indicating that every parity block depends on two neighboring data blocks:

$$\mathbf{C} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \tag{14}$$

The offset matrix $\mathbf{F}$ is a permutation matrix that rotates a vector by δ positions. For maximum fault tolerance we choose $\delta = 2$ (i.e., $\delta = R_{\max} - 1 = \lfloor 6/2 \rfloor - 1 = 2$):

$$\mathbf{F} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{15}$$

Multiplying $\mathbf{C}$ and $\mathbf{F}$ gives the parity-generation matrix $\mathbf{G} = \mathbf{CF}$:

$$\mathbf{G} = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}. \tag{16}$$

Each row of $\mathbf{G}$ specifies one parity equation. For instance,

$$P_1 = B_3 \oplus B_4, \quad P_2 = B_4 \oplus B_5, \quad P_3 = B_5 \oplus B_6,$$

$$P_4 = B_1 \oplus B_6, \quad P_5 = B_1 \oplus B_2, \quad P_6 = B_2 \oplus B_3.$$

The complete stripe is therefore $S = \{B_1, \ldots, B_6, P_1, \ldots, P_6\}$.

4.2. Node placement

Following the rule $(B_i, P_i) \to N_i$, each node stores one data block and one parity block. For $\delta = 2$, the placement is:

Node	Stored Blocks
N_1	$(B_1, P_1) \mid (B_1, P_{3,4})$
N_2	$(B_2, P_2) \mid (B_2, P_{4,5})$
N_3	$(B_3, P_3) \mid (B_3, P_{5,6})$
N_4	$(B_4, P_4) \mid (B_4, P_{6,1})$
N_5	$(B_5, P_5) \mid (B_5, P_{1,2})$
N_6	$(B_6, P_6) \mid (B_6, P_{2,3})$

(17)

This layout eliminates the one-block-per-node (OBPN) constraint and uses only $k = 6$ nodes to store $n = 12$ blocks.

4.3. Repair illustration

Single-node failure. Suppose node N_6 fails, losing (B_6, P_6). Block B_6 can be repaired via either of two local paths:

$$p_r^{(1)}(6) : (B_5, P_3) \quad \text{or} \quad p_r^{(2)}(6) : (B_1, P_4). \tag{18}$$

At the same time, parity P_6 can be re-encoded from (B_2, B_3). Both steps require only two helper blocks each, yielding $\rho_{\text{ZJC}} = 2$.

Multiple-node failure. If three nodes (N_2, N_3, N_4) fail simultaneously, some lost blocks (e.g., B_3) cannot be repaired immediately because their repair paths contain other failed symbols. ZJC performs *sequential repair*: reconstruct the available blocks first (e.g., B_2, B_4), then iteratively unlock the remaining ones. This asynchronous approach balances bandwidth and recovery latency. When paths are disjoint, repairs proceed in parallel.

4.4. Properties

The $ZJC(6)$ example demonstrates several important properties:

- Each data block participates in two distinct local groups, providing two independent recovery paths.
- The repair bandwidth is constant ($\rho = 2$) and independent of the stripe size k.
- The system tolerates up to $R_{\max} = \lfloor k/2 \rfloor$, in this case $R_{max} = 3$, node failures without global decoding.
- The storage overhead is minimized: $\gamma = 2$ but only $\vartheta = 6$ nodes are required, yielding $\eta = \vartheta \cdot \gamma = 12$.

Overall, $ZJC(6)$ confirms that the proposed design achieves fully local repair with low complexity and high availability, while reducing both node count and storage cost compared with RS and LRC codes.

5. Discussion

5.1. Why does ZJC place data and parity blocks together?

Placing data and parity blocks together does not imply that they cannot be stored separately. Our primary objective is to optimize storage costs and computational overhead. As mentioned in the introduction, adding extra nodes incurs substantial costs beyond mere disk expenses. This consideration led us to incorporate the number of nodes into the storage overhead equation.

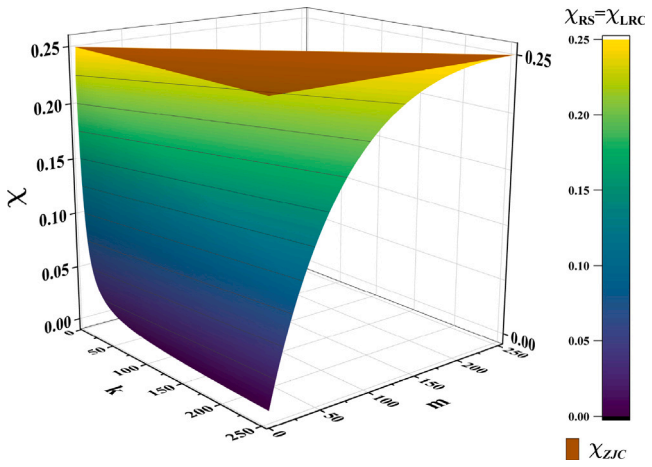


Fig. 6. Performance Comparison Between ZJC (brown plane), RS, and LRC ($\chi_{RS} = \chi_{LRC}$).

Our placement strategy reduces the number of storage nodes required by half compared to other coding methods with the same redundancy rate, effectively lowering storage costs. Moreover, the block placement in ZJC is carefully structured and governed by the offset parameter δ. Logically, δ is determined by the maximum recoverable node count $R_{\max}$, which in turn depends on the stripe size k. Consequently, all implicit design parameters are ultimately derived from a single visible parameter k. In traditional RS codes and LRC, co-locating data and parity blocks often impairs recoverability. However, our approach ensures recoverability through the parameter δ, enhancing fault tolerance while reducing storage overhead.

5.2. What is the advantage of double redundancy?

Fault tolerance is the primary objective of erasure-code design, preceding secondary concerns such as storage cost and computational efficiency. In fully local repair codes, global repair is intentionally eliminated, which requires each data block to be protected by multiple independent local encoding operations to ensure availability under node failures. A single local encoding is insufficient, as its repair conditions are more fragile, whereas two independent local encodings constitute the minimum practical configuration for reliable fully local repair. Although two local encodings would naturally introduce two parity blocks per data block ($\gamma = 3$), ZJC employs a local-group interleaving checking mechanism that constructs each parity block from two adjacent data blocks in a circular manner, allowing redundancy to be shared across the stripe. As a result, ZJC provides two independent local repair paths per data block with only one parity block per data block on average, yielding $\gamma = 2$, which is the minimum redundancy enabling recovery from up to $\lfloor k/2 \rfloor$ node failures under fully local repair constraints. While $\gamma = 2$ appears high compared with general erasure codes, high redundancy does not necessarily imply high storage overhead. By introducing a storage-overhead metric η that incorporates node provisioning cost, we show that ZJC achieves optimal storage efficiency. Compared with replication, which offers simple recovery but limited fault tolerance unless excessive redundancy is used, and conventional erasure codes, which provide higher tolerance at the cost of global repair and high latency, ZJC achieves erasure-code-level fault tolerance under the same redundancy budget while maintaining lower storage overhead than replication for the same level of reliability, positioning $\gamma = 2$ as a competitive and well-justified operating point.

To quantify the advantage, we introduce a metric χ, which reflects the trade-off between fault tolerance and storage overhead. Formally, it measures the fault-tolerance gain per unit storage overhead. Let R_{max} denote the maximum number of node failures tolerated and η represent the storage overhead. Then, χ is directly proportional to R_{max} and inversely proportional to η:

$$\chi = \frac{R_{max}}{\eta} \tag{19}$$

For ZJC, we obtain $\chi_{ZJC} = 0.25$ (a constant), while for RS and LRC, $\chi_{RS} = \chi_{LRC} = \frac{km}{(k+m)^2}$, where k is the number of data blocks and $m = n-k$ is the number of parity blocks. Detailed derivations are provided in the *Appendix*.

By evaluating all possible cases, we derive Fig. 6. Observing the figure, we conclude that ZJC (represented by the brown plane) consistently achieves the best results. Moreover, we prove in the *Appendix* that $\chi_{RS} = \chi_{LRC} \leq \chi_{ZJC}$, confirming that ZJC achieves a well-balanced trade-off between fault tolerance and storage efficiency.

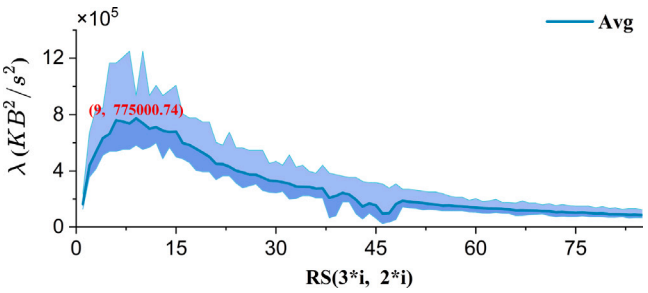


Fig. 7. Preliminary Tests.

6. Evaluation

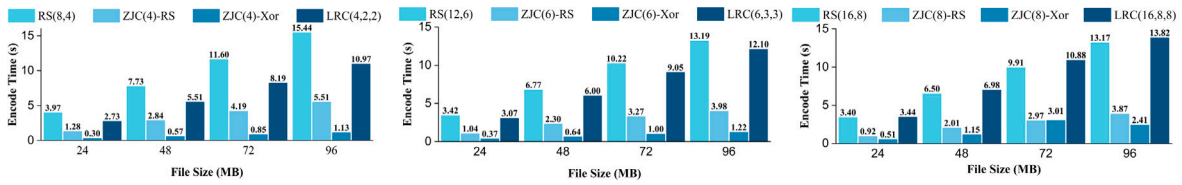
In the previous sections, we discussed the encoding and storage approaches and provided the RS-based and XOR-based implements for ZJC. This section will assess ZJC by comparing it with RS and LRC. Furthermore, we will present the actual performance of ZJC in the distributed storage system. The results show that ZJC has made a more significant improvement over RS and LRC, and demonstrating the feasibility of ZJC. Our codes are available at Github: https://github.com/Zijian-Zhou/ZJC_Compare.

6.1. Experimental environment

There are two experimental environments because we are involved in computing the comparison of distinct codes in localhost and actual performance in DSS.

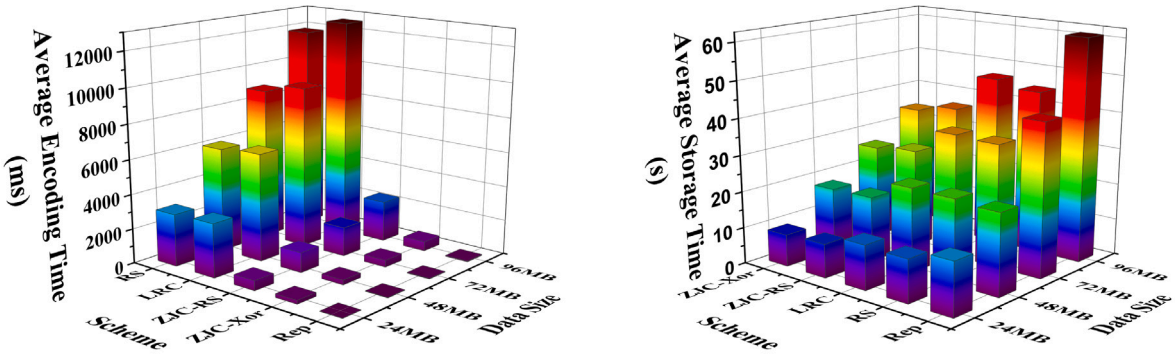
(env1). The different codes comparison uses this $env1$, and we implement RS, RS-based ZJC, XOR-based ZJC, and LRC in $env1$. In $env1$, we set the same redundancy rate for those four erasure codes and build three comparison groups, i.e. $[RS(8,4), ZJC(4)\_RS, ZJC(4)\_XOR, LRC(4,2,2)]$, $[RS(12,6), ZJC(6)\_RS, ZJC(6)\_XOR, LRC(6,3,3)]$ and $[RS\ (16,8), ZJC(8)\_RS, ZJC(8)\_XOR, LRC(16,4,4)]$. We tested the encoding and repair performance for each group in sizes of $24MB$, $48MB$, $72MB$, and $96MB$. We guarantee the failure is recoverable for each code and set the block lost from 1 block to k blocks. The experimental machine has the Intel Core i7-11700 @ 2.50 GHz ×16 CPU and $16GB$ Memory. We run the experiment on 64-bits Ubuntu 22.04.4 LTS.

(env2). The $env2$ comprises 16 cloud ECS (Elastic Compute Service) servers as a storage network. The OS of these 16 nodes is 64-bits Ubuntu 22.04.2 LTS. The CPU is a Single Core of Intel(R) Xeon (R) Gold 6278C @ 2.60 GHz. Bandwidth is $1\ Mbits/s$, and memory is $2GB$. The client is a 64-bits Ubuntu 22.04.4 LTS machine with an Intel Core i7-11700 @ 2.50 GHz ×16 CPU and $16GB$ Memory. We built a lightweight distributed storage system following the read and write processes of HDFS, where we complete our experiments. In this environment, we deployed $[Replication(1), RS(16,8), ZJC\ \ (8)\_RS, ZJC(8)\_XOR, LRC(16,4,4)]$.



(a) $k = 4$ (b) $k = 6$ (c) $k = 8$

Fig. 8. The encoding performance comparison in *env1*.



(a) Average Encoding Time (b) Average Storage Time

Fig. 9. The Encoding and Storage Analysis in *env2* under $k = 8$.

Where *Replication(1)* presents one more extra copies. In addition, we also tested the encoding and repair performance for each group in sizes of $24MB$, $48MB$, $72MB$, and $96MB$. In our experiments, node failures were simulated by directly shutting down the distributed storage service running on the node. A node was deemed faulty when the client could no longer access it. Under the same redundancy rate, ZJC utilizes only 8 nodes, which is half the number used by RS and LRC, leaving 8 of the 16 nodes unused. In our experiment, we ensure all schemes remain decodable during fault simulation. For instance, we simulated failures of 8 nodes, for ZJC, 4 of these failed nodes were unused and thus had no impact on its performance. However, the failure of the other 4 nodes resulted in the loss of half of ZJC's data blocks, aligning its failure scenario with that of the other EC schemes. Thus, the number of lost data blocks remains comparable across schemes.

6.2. Preliminary experiment

The computation of RS codes' coding and decoding processes entails operations within the Galois domain, wherein the various configurations of RS code parameters profoundly influence the resulting coding and decoding rates. Section 3.4 gives the $RS(n,k)$ settings for the RS-based ZJC scheme must adhere strictly to the prescribed $n : k : m = 3 : 2 : 1$ ratio. In the pursuit of optimal RS coding configurations, this section meticulously analyzes the variable $\lambda = v_{ec} \cdot v_{dc}$, with the utmost value, $\lambda_{max} = max(\lambda_i)$, serving as the fundamental RS setting for subsequent experimental iterations. Herein, v_{ec} and v_{dc} denote, respectively, the coding rate and decoding rate of the RS code, forming integral components of this intricate analysis.

We tested the total size of $20MB$ data delineated in Fig. 7, the pinnacle λ_{max}^{env1} identified as λ_9. Consequently, to optimize the RS coding and decoding rates, the RS setup parameters for all subsequent experiments are established as $RS(27,18)$, ensuring the swiftest performance in both encoding and decoding processes.

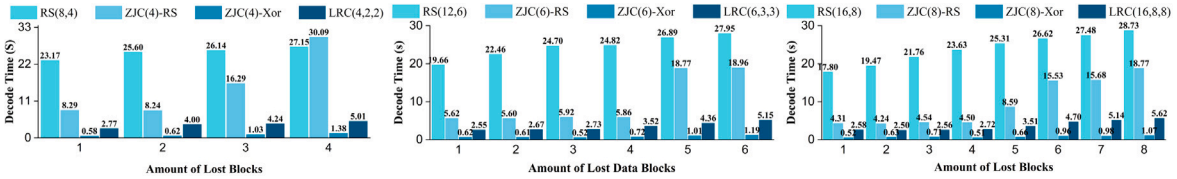
6.3. ZJC performance

6.3.1. Encoding performance

Fig. 8 shows the encoding performance for ZJC rather than RS and LRC. The second and third columns are RS-based and XOR-based for ZJC. It shows that the encoding time consumption of ZJC is much less than that of RS and LRC. ZJC RS-based improves the efficiency 67.91% than RS on average, and XOR-based is 87.5%. Compared to LRC, on average, RS-based enhances 62.32% and XOR-based advances 86.51%. These results prove the decrement of computation complexity by removing the global group and calculating in the totally local groups. In terms of different implements of ZJC, XOR-based consumption is 59.67% than the RS-based. The performance of XOR-based ZJC indicates again that erasure recovery by XOR operation is much faster than finite-field calculation. Our proposition is that using XOR for ZJC reduces recovery delay. The detailed statistics are presented in *Appendix* due to the length limitation.

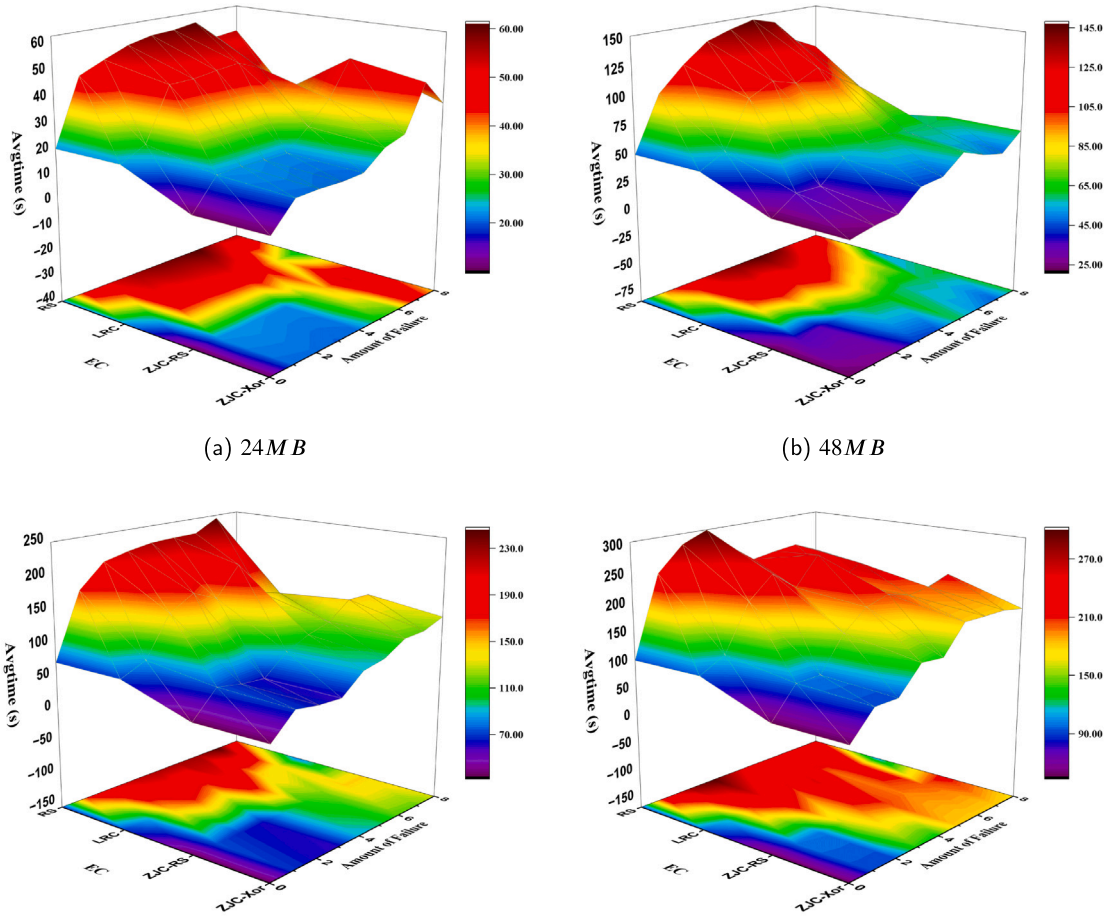
Fig. 9 presents the average encoding and storage costs for different recovery strategies in *env2*. In order for the data bars not to be obscured, we have adjusted the order of the schemes in coordinates in subgraphs Fig. 9(a) and Fig. 9(b). From the perspective of coding efficiency, it is evident that RB is always the fastest, as it involves no computation. Among all EC coding schemes, the ZJC consistently demonstrates the highest efficiency. Notably, the XOR-based ZJC code achieves optimal efficiency across all scenarios. The encoding results in *env2* are largely consistent with those in *env1*, further highlighting the high efficiency of the ZJC code. Numerical comparisons reveal that ZJC-RS is on average 20.54% and 22.33% faster than RS and LRC with the same redundancy, respectively. Similarly, ZJC-XOR outperforms RS, LRC, and ZJC-RS by 24.20%, 25.90%, and 4.59%, respectively. For detailed data, please refer to the *Appendix*.

In terms of overall storage efficiency, as Fig. 9(b) depicted, the differences among all EC schemes are minimal. While the coding efficiency of RS and LRC is lower than that of ZJC, under bandwidth-constrained conditions, such as in *env2*, where each node has a bandwidth limit of $1Mbit/s$, they achieve higher efficiency by storing data in parallel across 16 nodes compared to ZJC, which stores data in parallel across only 8 nodes. This characteristic is also reflected in RB scheme, where data is stored on only 2 nodes simultaneously, resulting in significantly higher time consumption than other EC schemes. However, as the data volume increases, the parallelism benefits of RS and LRC fail to offset their computational overhead. Consequently, the performance gap between these schemes and ZJC becomes increasingly evident, further underscoring the efficiency of our codes.



(a) $k = 4$ (b) $k = 6$ (c) $k = 8$

Fig. 10. The recovery of $24MB$ data in *env1*.



(a) $24MB$ (b) $48MB$

(c) $72MB$ (d) $96MB$

Fig. 11. The recovery result in *env2*.

6.3.2. Repair of multiple blocks lost

We compare the recovery delay for each group, ensuring all the blocks' lost situation is repairable and that up to k failure is repairable. Fig. 10 presents distinct recovery time consumptions at the data size of $24MB$ for $k = 4$, $k = 6$, and $k = 8$ for RS, LRC, and ZJC. ZJC XOR-based is still the best solution in the four methods, and most of the circumstances finished in $1s$, whatever the number of blocks lost occurred. Although the RS-based ZJC is faster than LRC, it dramatically improves the RS code. There is only one point that RS-based ZJC is slower than RS code, i.e., there are 4 blocks lost when $k = 4$ at the data size is $24MB$, as Fig. 10(a) depicted. This case is caused by localhost calculating without considering the data transfer delay in the network. ZJC must satisfy the condition of (11) or (12). Thus, there are some accumulated delays for multiple local groups. Although single local group repair is faster than RS global repair, it leads to slight consumption resulting from accumulated delay and the sum of all local group recovery time consumption. However, a realistic storage system will eliminate the slight consumption because ZJC has low repair bandwidth and can access blocks in a suitable time, guaranteeing faster repair than RS code.

In the recovery pattern, RS-based and XOR-based ZJC have averagely improved the efficiency by 53.87% and 96.5% than RS code, respectively. ZJC XOR-based has enhanced 77.5% and 90.83% than LRC and ZJC RS-based. The results of other groups and the detailed statistics are presented in *Appendix* due to the length limitation.

The Fig. 11 illustrates the recovery efficiency of distinct approaches in the *env*2. Since RB with the same redundancy rate can tolerate only a single node failure, and our discussion focuses on scenarios involving multiple node failures, RB is excluded from our analysis. It can be observed that ZJC maintains the shortest recovery time, i.e., the highest recovery efficiency, in nearly all failure scenarios. However, as shown in the subplots, ZJC's recovery efficiency under half-node failures is lower than that of LRC, despite its superior local computation efficiency compared to LRC. As a result, in the surface plot, LRC exhibits a small

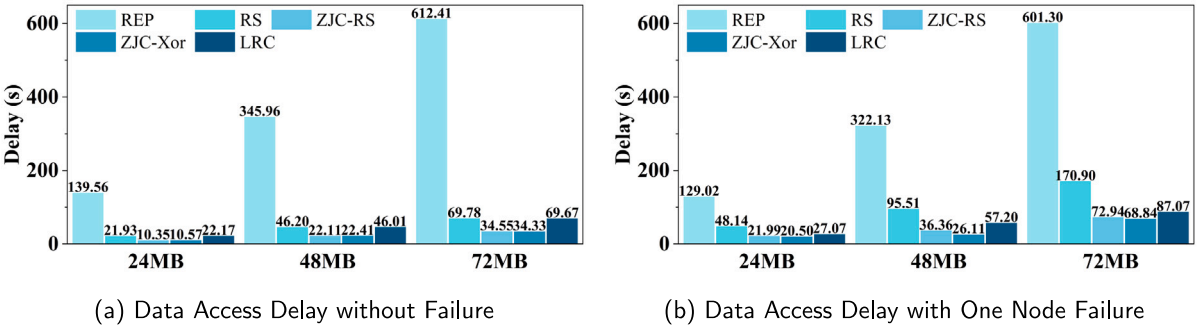


Fig. 12. Data Access Delay Analysis in *env2*.

valley under half-node failures, which corresponds to a lighter-colored area in the projection plane. This outcome arises because, during actual storage operations, when the client attempts to retrieve data blocks from all 8 DataNodes storing the data, it promptly detects node failures due to the lack of response from the DataNodes. Once all 8 DataNodes are identified as faulty, RS and LRC simultaneously retrieve all parity blocks from the remaining 8 healthy nodes to initiate global recovery. Given the higher computational complexity of RS compared to LRC, LRC achieves higher efficiency than RS. For ZJC, only 4 out of the 8 faulty nodes impact its operation, as the remaining 8 healthy nodes include 4 that store both data and parity blocks. Consequently, recovery proceeds asynchronously for ZJC, which is less efficient than LRC's one-time synchronous recovery. Nevertheless, ZJC still outperforms RS in efficiency. It is worth noting that failures involving half of the nodes are relatively rare, most failures involve a small number of individual nodes. This further underscores ZJC's high practicality and efficiency in real-world applications.

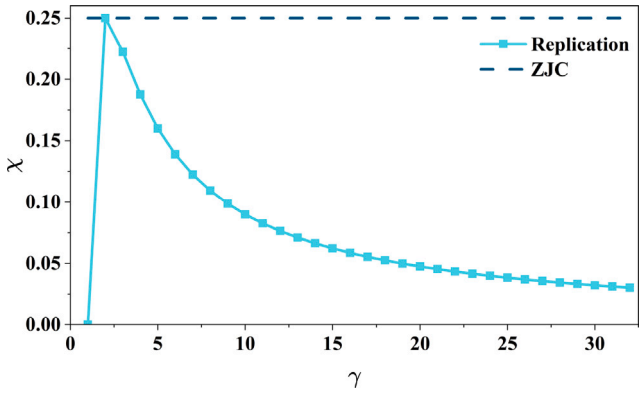


Fig. 13. The Evaluation of χ for Replication.

6.4. Comparison with replication

In Section 6.3.2, replication was not included in the recovery-delay comparison, since direct comparison between fundamentally different techniques may appear unfair, and replication cannot tolerate as many failures as erasure codes under the same redundancy. In this Section, to provide a more comprehensive and practical comparison.

6.4.1. Data access latency

We revisited the original experimental data collected in *env2* and added replication-based results, as shown in Fig. 12(a) and Fig. 12(a). Under $\gamma = 2$, the replication can only recover one node failure. As a result, we only compare different approaches in two cases, i.e., without failure and one node failure. The results show that replication consistently exhibits the highest access delay, mainly because all replicas of a block are stored on the same node, leading to bandwidth bottlenecks. In contrast, erasure-coded systems distribute blocks across multiple nodes, achieving better load balancing and lower access latency. Despite the additional computation involved, erasure codes outperform replication in access performance. Moreover, ZJC further reduces access latency due to its simpler local encoding and repair operations.

6.4.2. Fault tolerance gain per unit storage overhead

Replication achieve high availability through compromising storage overhead. For a R-replicas system, it tolerates up to $R-1$ node failures through $R = \gamma$ replicas. Therefore, the metric χ for replication can be presented as

$$\chi_{Rep} = \frac{\gamma - 1}{\gamma^2}. \tag{20}$$

To provide a clear illustration, Fig. 13 depicts the variation of the fault-tolerance efficiency metric χ_{Rep} for replication and compares it with ZJC. As shown in the figure, replication achieves its maximum χ only at $R = \gamma = 2$, where it can tolerate at most one node failure in the worst case. Beyond this point, increasing redundancy leads to a monotonic decrease in the fault-tolerance gain per unit storage overhead. This behavior indicates that, although replication incurs low repair bandwidth due to its simple repair mechanism, it fails to achieve the desirable property of high fault tolerance with low storage overhead, as evidenced by the diminishing χ value. In contrast, ZJC use low redundance $\gamma = 2$ while achieving high tolerance up to half node failures.

6.5. Sensitivity analysis

To evaluate the robustness of the proposed ZJC scheme under realistic deployment settings, we conduct a sensitivity analysis with respect to the number of data blocks k, the redundancy ratio γ, and the number of global parity blocks g. The analysis focuses on how variations in these parameters affect both the encoding performance and the recovery performance.

In the experiments, the data size is fixed at 24 MB. We vary the number of data blocks as $k \in \{4, 6, 8, 10\}$ and consider different redundancy configurations by setting the number of global parity blocks to $g \in \{2, 3, 4\}$. All feasible parameter combinations under these settings are evaluated. For each configuration, we measure the encoding time and the recovery time under block failure scenarios with the number of lost blocks $f \in \{1, 2, 3, 4\}$.

Fig. 14 presents the encoding time under all redundancy settings, covering a total of 26 configurations. The results show that both RS-based and XOR-based ZJC consistently achieve superior encoding efficiency and exhibit no clear linear dependency on the parameter k. In contrast, the encoding performance of LRC is more sensitive to variations in γ, and generally degrades as k and g increase. Notably, the best encoding performance is observed around $k = 8$, which can be attributed to a balanced level of I/O parallelism. When k is

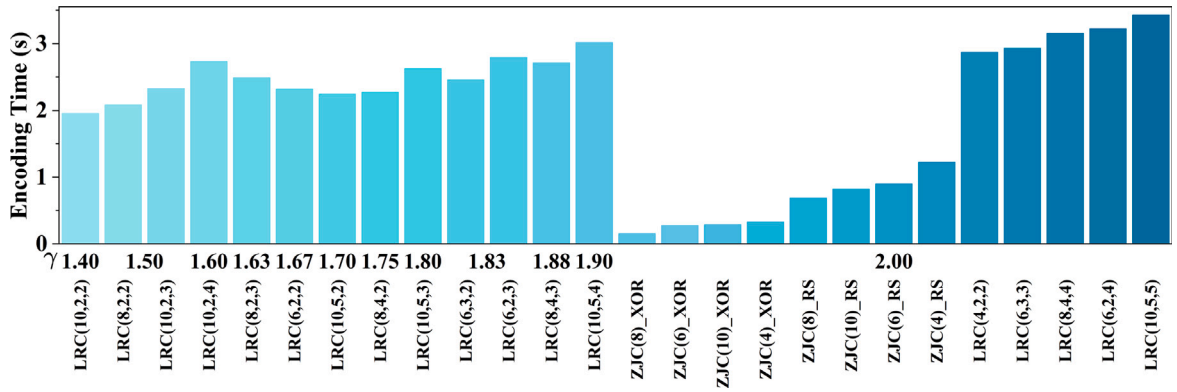


Fig. 14. Encoding Efficiency Sensitivity Analysis of γ.

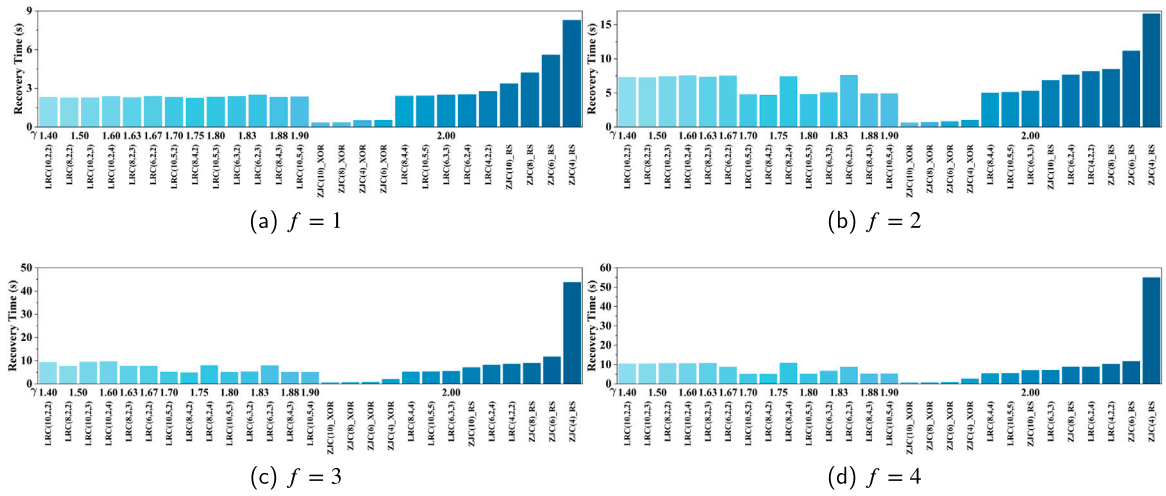


Fig. 15. Recovery Efficiency Sensitivity Analysis of γ.

small, limited parallelism prevents full utilization of I/O resources, whereas overly large k introduces excessive parallelism and associated overhead. In addition, the simplified computation structure of ZJC further contributes to its stable and efficient encoding behavior. These results indicate that ZJC maintains high encoding efficiency even under slightly higher redundancy, which is beneficial for practical systems. Fig. 15(a) to Fig. 15(d) report the recovery time under one to four block failures. The overall trends are consistent with previous experiments. XOR-based ZJC demonstrates the lowest recovery latency across all settings, often outperforming other schemes by several times. For multiple-block failure scenarios, RS-based ZJC with larger k values (e.g., $k = 10$) achieves better recovery performance than some LRC configurations with lower redundancy. However, RS-based ZJC generally exhibits higher recovery latency than XOR-based ZJC, mainly due to the computational complexity of Reed–Solomon decoding and limited I/O parallelism. This observation suggests that RS-based ZJC is more suitable for tolerance-oriented deployments with wider stripes, whereas XOR-based ZJC is preferable in efficiency-oriented environments.

To further isolate the impact of redundancy design, we group the configurations by the number of global parity blocks and compare cases with $g = 2$, $g = 3$, and ZJC ($g = 0$). We move the figures into the Appendix. Fig. a-4 to Fig. a-6 of Appendix show that encoding performance is only mildly affected by the number of global parity blocks. Similarly, Fig. a-7 to Fig. a-18 of Appendix indicate that recovery performance is not highly sensitive to variations in g. Importantly, configurations with higher redundancy and smaller local groups achieve better recovery performance under multiple-block failures. This observation implies that reducing local group size while increasing the number of local groups can effectively accelerate recovery, which further validates the design principles of ZJC.

7. Summary

In this paper, we have proposed ZJ-Codes (ZJC), a new class of erasure codes that achieve lower storage overhead and higher repair efficiency through a fully local repair mechanism. In addition, we introduced a novel storage-overhead equation that incorporates both redundancy and the economic cost of node provisioning, enabling a more comprehensive evaluation of system efficiency. Comparative analysis under the same redundancy ratio demonstrates that ZJC significantly reduces storage cost while maintaining strong fault tolerance compared with conventional RS and LRC schemes. We further presented a unified construction framework and provided both RS-based and XOR-based implementations, showing substantial performance gains in encoding and repair efficiency. Experimental results and prototype deployment in a distributed storage environment validate the practicality and effectiveness of ZJC. Looking ahead, we plan to conduct deeper theoretical analysis on the coding structure and fault-tolerance bounds of ZJC, and explore its integration into real-world large-scale storage systems. In the future, we will explore more advanced ZJC construction with less redundancy. We believe that the proposed ZJC offers a promising direction for next-generation erasure-coded storage, and we hope it will inspire further investigation and industrial adoption in hyperscale data centers.

CRediT authorship contribution statement

Zijian Zhou: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Xiaoheng Deng:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Data curation. **Xinjun Pei:** Writing – review & editing, Visualization, Formal analysis. **Yunlong Zhao:** Writing – review & editing. **Yurong Qian:** Writing – review & editing, Supervision. **Shaohua Wan:** Writing – review & editing, Supervision. **Kaiping Xue:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We would like to thank anonymous reviewers and editors for their comments. This work was supported by the National Natural Science Foundation of China Project (62172441, 62172449), the Joint Funds for Railway Fundamental Research of National Natural Science Foundation of China (Grant No. U2368201), special fund of National State Key Laboratory of Ni&Co Associated Minerals Resources Development and Comprehensive Utilization (GZSYS-KY-2024-073), Key Project of Shenzhen City Special Fund for Fundamental Research (202208183000751), the National Natural Science Foundation of Hunan Province (2023JJ30696), the High Performance Computing Center of Central South University.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.sysarc.2026.103735.

Data availability

Data will be made available on request.

References

[1] X. Zou, W. Xia, P. Shilane, H. Zhang, X. Wang, The design of a high-performance fine-grained deduplication framework for backup storage, IEEE Trans. Parallel Distrib. Syst. 36 (2025) 945–960, http://dx.doi.org/10.1109/TPDS.2025.3551306.

[2] D. Liu, X. Zou, T. Lu, P. Shilane, W. Xia, W. Huang, Y. Pan, H. Huang, Garbage collection does not only collect garbage: Piggybacking-style defragmentation for deduplicated backup storage, in: Proceedings of the Twentieth European Conference on Computer Systems, Association for Computing Machinery, New York, NY, USA, 2025, pp. 1026–1043, http://dx.doi.org/10.1145/3689031.3717493.

[3] Z. Shen, Y. Cai, K. Cheng, P.P.C. Lee, X. Li, Y. Hu, J. Shu, A survey of the past, present, and future of erasure coding for storage systems, 21, 2025, http://dx.doi.org/10.1145/3708994.

[4] Y. Cai, S. Lin, Z. Shen, J. Yang, J. Shu, Chameleonec: Exploiting tunability of erasure coding for low-interference repair, in: 2025 IEEE International Symposium on High Performance Computer Architecture, HPCA, 2025, pp. 15–28, http://dx.doi.org/10.1109/HPCA61900.2025.00013.

[5] I.S. Reed, G. Solomon, Polynomial codes over certain finite fields, J. Soc. Ind. Appl. Math. 8 (1960) 300–304, http://dx.doi.org/10.1137/0108018.

[6] Y. Ren, Y. Ren, X. Li, Y. Hu, J. Li, P.P.C. Lee, ELECT: Enabling erasure coding tiering for LSM-tree-based storage, in: 22nd USENIX Conference on File and Storage Technologies, FAST 24, USENIX Association, Santa Clara, CA, 2024, pp. 293–310, https://www.usenix.org/conference/fast24/presentation/ren.

[7] C. Huang, H. Simitci, Y. Xu, A. Ogus, B. Calder, P. Gopalan, J. Li, S. Yekhanin, Erasure coding in windows azure storage, in: 2012 USENIX Annual Technical Conference, USENIX ATC 12, USENIX Association, Boston, MA, 2012, pp. 15–26, https://www.usenix.org/conference/atc12/technical-sessions/presentation/huang.

[8] P. Gopalan, C. Huang, H. Simitci, S. Yekhanin, On the locality of codeword symbols, IEEE Trans. Inform. Theory 58 (2012) 6925–6934, http://dx.doi.org/10.1109/TIT.2012.2208937.

[9] D.S. Papailiopoulos, A.G. Dimakis, Locally repairable codes, IEEE Trans. Inform. Theory 60 (2014) 5843–5855, http://dx.doi.org/10.1109/TIT.2014.2325570.

[10] M. Makovenko, M. Cheng, C. Tian, Revisiting the optimization of cauchy reed-solomon coding matrix for fault-tolerant data storage, IEEE Trans. Comput. 71 (2022) 1839–1846, http://dx.doi.org/10.1109/TC.2021.3110131.

[11] C. Liu, Q. Wang, X. Chu, Y.W. Leung, G-crs: Gpu accelerated cauchy reed-solomon coding, IEEE Trans. Parallel Distrib. Syst. 29 (2018) 1484–1498, http://dx.doi.org/10.1109/TPDS.2018.2791438.

[12] N. Prakash, V. Lalitha, S.B. Balaji, P. Vijay Kumar, Codes with locality for two erasures, IEEE Trans. Inform. Theory 65 (2019) 7771–7789, http://dx.doi.org/10.1109/TIT.2019.2934124.

[13] X. Kong, X. Wang, G. Ge, New constructions of optimal locally repairable codes with super-linear length, IEEE Trans. Inform. Theory 67 (2021) 6491–6506, http://dx.doi.org/10.1109/TIT.2021.3103330.

[14] T. Niu, M. Lyu, W. Wang, Q. Li, Y. Xu, Cerasure: Fast acceleration strategies for xor-based erasure codes, in: 2023 IEEE 41st International Conference on Computer Design, ICCD, 2023, pp. 535–542, http://dx.doi.org/10.1109/ICCD58817.2023.00088.

[15] W. Wang, M. Lyu, T. Niu, Q. Li, L. Xu, Y. Xu, Fast acceleration strategies for xor-based erasure codes, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. (2024) http://dx.doi.org/10.1109/TCAD.2024.3432648, 1–1.

[16] S. Li, Q. Cao, S. Wan, W. Xia, C. Xie, gppm: A generalized matrix operation and parallel algorithm to accelerate the encoding/decoding process of erasure codes, ACM Trans. Arch. Code Optim. 20 (2023) http://dx.doi.org/10.1145/3625005.

[17] Y. Uezato, Accelerating xor-based erasure coding using program optimization techniques, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, Association for Computing Machinery, New York, NY, USA, 2021, http://dx.doi.org/10.1145/3458817.3476204.

[18] L. Xu, M. Lyu, Z. Li, Y. Li, Y. Xu, Deterministic data distribution for efficient recovery in erasure-coded storage systems, IEEE Trans. Parallel Distrib. Syst. 31 (2020) 2248–2262, http://dx.doi.org/10.1109/TPDS.2020.2987837.

[19] L. Xu, M. Lyu, Z. Li, C. Li, Y. Xu, A data layout and fast failure recovery scheme for distributed storage systems with mixed erasure codes, IEEE Trans. Comput. 71 (2022) 1740–1754, http://dx.doi.org/10.1109/TC.2021.3105882.

[20] G. Zhang, Z. Huang, X. Ma, S. Yang, Z. Wang, W. Zheng, RAID+: Deterministic and balanced data distribution for large disk enclosures, in: 16th USENIX Conference on File and Storage Technologies, FAST 18, USENIX Association, Oakland, CA, 2018, pp. 279–294, https://www.usenix.org/conference/fast18/presentation/zhang.

[21] C. Liu, Q. Wang, X. Chu, Y.W. Leung, H. Liu, Esetstore: An erasure-coded storage system with fast data recovery, IEEE Trans. Parallel Distrib. Syst. 31 (2020) 2001–2016, http://dx.doi.org/10.1109/TPDS.2020.2983411.

[22] Y. Hu, L. Cheng, Q. Yao, P.P.C. Lee, W. Wang, W. Chen, Exploiting combined locality for Wide-Stripe erasure coding in distributed storage, in: 19th USENIX Conference on File and Storage Technologies, FAST 21, USENIX Association, 2021, pp. 233–248, https://www.usenix.org/conference/fast21/presentation/hu.

[23] S. Lin, G. Gong, Z. Shen, P.P.C. Lee, J. Shu, Boosting full-node repair in erasure-coded storage, in: 2021 USENIX Annual Technical Conference, USENIX ATC 21, USENIX Association, 2021, pp. 641–655, https://www.usenix.org/conference/atc21/presentation/lin.

[24] X. Li, K. Cheng, K. Tang, P.P.C. Lee, Y. Hu, D. Feng, J. Li, T.Y. Wu, ParaRC: Embracing Sub-Packetization for repair parallelization in MSR-Coded storage, in: 21st USENIX Conference on File and Storage Technologies, FAST 23, USENIX Association, Santa Clara, CA, 2023, pp. 17–32, https://www.usenix.org/conference/fast23/presentation/li-xiaolu.

[25] J. Gao, J. Shu, B. Yan, Y. Zhang, K. Huang, Stripeless data placement for $\{Erasure-Coded\}\{In-Memory\}$ storage, in: 19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 25, 2025, pp. 821–838.

[26] J. Zhang, S. Li, X. Liao, X. Liu, Aggregation decoding for multi-failure recovery in erasure-coded storage, in: 2014 IEEE 17th International Conference on Computational Science and Engineering, 2014, pp. 1326–1331, http://dx.doi.org/10.1109/CSE.2014.253.

[27] X. Pei, Y. Wang, X. Ma, Y. Fu, F. Xu, Mcrtree: A mutually cooperative recovery scheme for multiple losses in distributed storage systems based on tree structure, in: 2014 9th IEEE International Conference on Networking, Architecture, and Storage, 2014, pp. 158–167, http://dx.doi.org/10.1109/NAS.2014.33.

[28] Q. Yu, L. Wang, Y. Hu, Y. Xu, D. Feng, J. Fu, X. Zhu, Z. Yao, W. Wei, Boosting multi-block repair in cloud storage systems with wide-stripe erasure coding, in: 2023 IEEE International Parallel and Distributed Processing Symposium, IPDPS, 2023, pp. 279–289, http://dx.doi.org/10.1109/IPDPS54959.2023.00036.

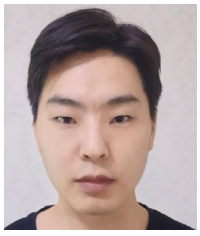
[29] Z. Shen, P.P.C. Lee, J. Shu, W. Guo, Cross-rack-aware single failure recovery for clustered file systems, IEEE Trans. Dependable Secur. Comput. 17 (2020) 248–261, http://dx.doi.org/10.1109/TDSC.2017.2774299.

[30] Z. Shen, J. Shu, P.P.C. Lee, Reconsidering single failure recovery in clustered file systems, in: 2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, 2016, pp. 323–334, http://dx.doi.org/10.1109/DSN.2016.37.

[31] J. Zhang, S. Li, X. Liao, Redu: reducing redundancy and duplication for multi-failure recovery in erasure-coded storages, J. Supercomput. 72 (2016) 3281–3296, http://dx.doi.org/10.1007/s11227-015-1397-9.
[32] Z. Shen, J. Shu, P.P.C. Lee, Y. Fu, Seek-efficient i/o optimization in single failure recovery for xor-coded storage systems, IEEE Trans. Parallel Distrib. Syst. 28 (2017) 877–890, http://dx.doi.org/10.1109/TPDS.2016.2591040.
[33] X. Li, K. Cheng, Z. Shen, P.P.C. Lee, Fast proactive repair in erasure-coded storage: Analysis, design, and implementation, IEEE Trans. Parallel Distrib. Syst. 33 (2022) 3400–3414, http://dx.doi.org/10.1109/TPDS.2022.3152817.
[34] C. Gan, Y. Hu, L. Zhao, X. Zhao, P. Gong, D. Feng, Revisiting network coding for warm blob storage, in: 23rd USENIX Conference on File and Storage Technologies, FAST 25, USENIX Association, Santa Clara, CA, 2025, pp. 139–154, https://www.usenix.org/conference/fast25/presentation/gan.
[35] V. Estrada Galiñanes, E. Miller, P. Felber, J.F. Pâris, Alpha entanglement codes: Practical erasure codes to archive data in unreliable environments, in: 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, 2018, pp. 183–194, http://dx.doi.org/10.1109/DSN.2018.00030.
[36] S. Kadekodi, S. Silas, D. Clausen, A. Merchant, Practical design considerations for wide locally recoverable codes (lrcs), 19, 2023, http://dx.doi.org/10.1145/3626198.
[37] O. Kolosov, G. Yadgar, M. Liram, I. Tamo, A. Barg, On fault tolerance, locality, and optimality in locally repairable codes, ACM Trans. Storage 16 (2020) http://dx.doi.org/10.1145/3381832.
[38] W. Song, K. Cai, C. Yuen, K. Cai, G. Han, On sequential locally repairable codes, IEEE Trans. Inform. Theory 64 (2018) 3513–3527, http://dx.doi.org/10.1109/TIT.2017.2711611.

Zijian Zhou received the B.S. degree from Hefei University in 2023. He is currently pursuing the M.S. degree at Xinjiang University and conducting cooperative research as a joint master's student at Central South University. His research interests include information security, distributed and decentralized systems, and security issues in machine learning.

Xiaoheng Deng received the Ph.D. degree in computer science from Central South University, Changsha, Hunan, P.R. China, in 2005. Since 2006, he has been an Associate Professor and then a Full Professor with the Department of Communication Engineering, Central South University. He is a Joint researcher of Shenzhen Research Institute, Central South University. He is a senior member of IEEE, a senior member of CCF, a member of CCF Pervasive Computing Council, and a member of ACM. He has been a chair of CCF YOCSEF CHANGSHA from 2009 to 2010. His research interests include network security, edge computing, Internet of Things, online social network analysis, data mining, and pattern recognization.



Xinjun Pei received the Ph.D. degree with the School of Computer Science and Engineering, Central South University, Changsha, China. He is currently an Associate Professor with the School of Software, Xinjiang University, Ürümqi, China. Since 2017, he has been engaged in the direction of information security. His research interests include deep learning, edge computing and IoT security. He has authored or coauthored more than 20 articles for scientific books, journals, and conferences. His research has been published with flagship information security and artificial intelligence venues, such as IEEE TIFS, IEEE TDSC, CVPR, IEEE TII, IEEE TNSE, and IEEE TNSM.

Yunlong Zhao received the bachelor's degree from the School of Computer Science, Beijing University of Posts and Telecommunications, in 2020. He is currently pursuing a Ph.D. degree with the school of Computer Science and Engineering, Central South University, China. Since 2021, he has been engaged in the direction of privacy protection. His research interests include federated learning, edge computing, and security issues in neural networks.

Yurong Qian received the B.S. and M.S. degrees in computer science and technology from Xinjiang University, Urumqi, China, in 2000, and the Ph.D. degree in Biology from Nanjing University, Nanjing, China, in 2010. She was a Postdoctoral Researcher with the Department of Electrical and Computer Engineering, Hanyang University, South Korea, from 2012 to 2013, and is currently a Professor with the School of Computer Science and Technology and the School of Software, Xinjiang University. Her research interests include computational intelligence, such as Big Data processing, image processing, and artificial neural networks. Dr. Qian is a Senior member of the Chinese Computer Society.

Shaohua Wan received the Ph.D. degree in computer science and technology from the College of Computer Science, Wuhan University, Wuhan, China, in 2010. He is currently a Full Professor with Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen, China. From 2016 to 2017, he was a Visiting Professor with the Department of Electrical and Computer Engineering, Technical University of Munich, Munich, Germany. He has authored 150 peer-reviewed research papers and books, including more than 40 IEEE/ACM transactions papers, such as IEEE TII, IEEE TITS, ACM TIT, IEEE TNSE, and many top conference papers in the fields of edge intelligence. His research focuses on deep learning for the Internet of Things.

Kaiping Xue (M'09-SM'15) received his bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC), in 2003 and received the Ph.D. degree from the Department of Electronic Engineering and Information Science (EEIS), USTC, in 2007. From May 2012 to May 2013, he was a postdoctoral researcher with the Department of Electrical and Computer Engineering, University of Florida. Currently, he is a Professor in the School of Cyber Security, USTC. His research interests include next-generation Internet architecture design, transmission optimization and network security. He serves on the Editorial Board of several journals, including the IEEE TDSC, the IEEE TWC, and the IEEE TNSM. He has also served as a (Lead) Guest Editor for many reputed journals/magazines, including IEEE JSAC, IEEE Communications Magazine, and IEEE Network. He is an IET Fellow and an IEEE Senior Member.